

C200 PROGRAMMING ASSIGNMENT № 1

FUNCTIONS & LEARNING ABOUT HOMEWORK

FALL 2024

Dr. M.M. Dalkilic

Computer Science

School of Informatics, Computing, and Engineering

Indiana University, Bloomington, IN, USA

September 14, 2024

The purpose of this homework is two-fold: (1) you'll develop your skills in implementing functions, formal and actual parameters, environment and scope (2) how homework will be delivered and submitted. Although the problems can be used for individuals, this course uses paired-programming. Please carefully read about pair-programming at the end of this document.

Please submit the completed HW (a1.py file) to Autograder and also push the file (completed a1.py) to your github repo by **11:00 PM, Friday, September 20**.

Instructions

The course uses an online system for grading the Python code. The system can be accessed at `c200.luddy.indiana.edu`, and you should login via your official IU username and password. Upon login, you can submit HW via 'Create Submission' button. Remember that you **must** push the HW files to GitHub and also submit to the Autograder before the HW deadline. If you do not see options to submit at the given URI then please retry the url in incognito mode (open chrome browser in incognito mode).

Autograder is generally opened to accept HW submissions a few days after the HW is released. If you don't see the relevant Assignment tab in the Autograder, please feel free to check back periodically. We will also announce on InScribe when we release the Autograder.

Paired-Programming

Homework is generally done with a randomly selected partner. In the last section, some information about contacting and communicating with this likely new student will be facilitated. **As always, all the work will be with only you and your partner; but *both* of you should contribute.** If your partner does not respond, you must complete the homework on your own.

You will not be assigned a new partner—this is infeasible for such a large class. Both students must submit their individual homework, since you are graded individually. Your grade may be different from your partner's grade, since there can be differences in code.

Finding your programming partner: You can find your programming partner by simple text search in this document i.e. use `ctrl + F` (for windows system) or `CMD + F` (for MACOS), and type your IU username, press enter. This will highlight the line having your username in the list of programming partners, please, simply look for the name adjacent to your's in the list.

You must complete this before **due date**. You will submit your work by pushing your code to your GitHub repository and by uploading your completed python file to the Autograder. Please remember that.

- You will **not** turn anything in on canvas.
- You do **not manually upload files** to your repository using GitHub's "Upload files" tool.
- You must submit your individual program even though you have a partner, since you'll be graded individually.
- Your github will accept pushes at *any* time. Once the deadline for homework is passed, if you push again, you'll rewrite over the correct time and the solution will be past the deadline and not graded. If your timestamp is 11:01PM or later, the homework will not be graded under almost any circumstance. So do not wait until 10:59PM to commit and push your changes. Pushing with only a few minutes left is **not** a valid reason to be graded.
- You cannot discuss homework outside of your partner, use tools like Chat-GPT, copy a solution—a person who cheats cheats himself.
- Struggling is part of any practiced skill—it's how we learn.

Remarks: (1) We do not print or return units—only values. If a function is calculating a percentage, dollar amount, *etc.* the percentage sign % is **not** shown, the dollar sign \$ is not shown. For the problem descriptions, we occasionally include them to make interpreting the problem easier. (2) Often numerical values will be infinite. Python will give you **by default** about 15 decimal places. We'll learn how to shorten these values later. For now, for example we'll write 104.17 for 104.16666666666667. On this HW, you are allowed to use `round()` function to round the final answer before returning it in the function—for example, if your final answer is 6.85457 then it's rounded equivalent is 6.85, and can be obtained by doing **`round(6.8547, 2)`**. The first parameter to the `round()` function is the actual value that you want to round, and the second argument is the number of places you want it rounded to. Sometimes, we don't want to round—we simply want the integer portion. Recalling from lecture we can write **`int(.)`**. For example, `int(1.5) = 1`.

Comments about Homework

You will be given the function signature without a body. For each problem, you'll replace the Python keyword **`pass`** with code. Do not change any signature (meaning do not change the

function name, function arguments/parameters). If you do, the autograder will fail and you'll receive a zero. If you add any extra functions, they must be local to the function you are building; otherwise, the autograder will fail.

Keyboard Quickies

To uncomment multiple lines (Windows):

1. select the lines to be commented
2. Ctrl+k+u

To comment multiple lines (Windows):

1. select the lines to be commented
2. Ctrl+/

To comment or uncomment multiple lines (Mac):

1. select the lines to be commented
2. cmd + /

To undo (Windows):

- Ctrl+Z

To undo (Mac)

- Command+Z

Some of these problems were taken or inspired by the excellent introductory texts *Applied Calculus* by Tan, 2005, *Thinking Mathematically* 5th ed. by Blitzer, 2011.

Problem 1: Volume of a Cone

The volume of a cone with radius r and height h is:

$$c(r, h) = \frac{1}{3}\pi r^2 h \quad (1)$$

For example, 2 cm radius and 5 cm height,

$$c(2, 5) \approx 20.94 \text{ cm}^3 \quad (2)$$

For 3 cm radius and 7 cm height

$$c(3, 7) \approx 65.97 \text{ cm}^3 \quad (3)$$

Deliverables Problem 1

- Complete the cone volume function in the file a1.py.
- You must use `math.pi` from the `math` module.
- Round the answer to 2 decimal places. Use `round(x,y)` to round x to y decimal places. For example,
- Example of rounding to different places after decimal are shown below.

```
1 >>> round(1.252, 2)  #rounds to two places
2 1.25
3 >>> round(1.252, 1)  #rounds to 1 place
4 1.3
5 >>> round(1.252, 0)  #rounds to 0 place
6 1.0
```

Problem 2: Oxygen Content of a Pond

The oxygen content t days after organic waste has been dumped into a pond is given by:

$$f(t) = 100 \frac{t^2 + 10t + 100}{t^2 + 20t + 100} \quad (4)$$

percent of its normal level. For example,

$$f(0) = 100 \quad (5)$$

$$f(10) = 75 \quad (6)$$

Deliverables Problem 2

- Complete the oxygen content function in the file a1.py.
- Round the returned value to 2 decimal places.

Problem 3: TV Viewing Patterns

According to A.C. Nielsen Co., the percent of U.S. households watching television during the weekdays (about a decade ago) starting at 4:00PM for eight hours is modeled as $P(t)$:

$$P(t) = 0.01354t^4 - 0.49375t^3 + 2.58333t^2 + 3.8t + 31.60704 \quad (7)$$

if $0 \leq t \leq 8$ where $t = 0$ corresponds to 4:00P. For example,

$$P(0) = 31.61\% \quad (8)$$

$$P(3) = 54.02\% \quad (9)$$

$$P(8) = 30.0\% \quad (10)$$

Deliverables Problem 3

- Complete tv percent function in the file a1.py.
- Round the answer to 2 decimal places.

Problem 4: Toxic Waste

A city's main well was recently found to be contaminated with trichloroethylene, a cancer-causing chemical, as a result of an abandoned chemical dump leaching chemicals into the water. A proposal submitted to city council members indicates that the cost, measured in millions of dollars to remove $x\%$ of the toxic pollutant is given by:

$$\text{cost}(x) = \frac{0.5x}{100 - x} \quad (11)$$

for $0 < x < 100$. For example, 50%, 70%, and 90% cost

$$\text{cost}(50) = \$5 \text{ million} \quad (12)$$

$$\text{cost}(70) \approx \$1.17 \text{ million} \quad (13)$$

$$\text{cost}(90) = \$4.5 \text{ million} \quad (14)$$

Deliverables Problem 4

- Complete the percent cost function in the file a1.py.
- Round the answer to 2 decimal places.

Problem 5: Cowling's Rule

Cowling's rule is a method for calculating pediatric drug dosages. If a denotes the adult dosage (in milligrams) and t is the age of the child (in years), then the child's dosage is given by:

$$D(t, a) = \frac{t+1}{24}a \quad (15)$$

For example, if $a = 500$ mg and $t = 4$ years, then

$$D(t, a) = 104.17 \text{ mg} \quad (16)$$

(17)

Deliverables Problem 5

- Complete Cowling's rule function in the file a1.py.
- Round the answer to 2 decimal places.

Problem 6: Flu Outbreak

During a flu outbreak in a school of 1000 children, the number of infected children, I , was expressed in terms of the number of susceptible (but still healthy) children, S , by:

$$I(S) = \lceil 192 \log_2\left(\frac{S}{762}\right) - S + 763 \rceil \quad (18)$$

For example,

$$I(100) = 101 \text{ students} \quad (19)$$

$$I(300) = 205 \text{ students} \quad (20)$$

Deliverables Problem 6

- The math module is imported for this problem.
- Complete the I function in the file a1.py.
- You may get negative values for some inputs values that you provide to the function, and that's fine.
- To remind you, $\lceil x \rceil = y$ is the smallest integer y such that $x \leq y$.
- Use **math.ceil()** from the math module discussed in lecture.

Problem 7: Average Cost

Let $C(q)$ be the cost of producing q items. A company's cost function is (in \$) given by

$$C(q) = 0.01q^3 - 0.6q^2 + 13q + 1000 \quad (21)$$

The **average cost** $A(q)$ is given by

$$A(q) = \frac{C(q)}{q} \quad (22)$$

Deliverables Problem 7

- Complete both functions C, A in the file a1.py
- For this problem, you'll need to make your own input/outputs

Problem 8: Sales Model

A company contracted by you has built a sales model that returns the number of plastic ducks sold ($\times 10^3$)

$$hh(t) = \left\lfloor \frac{532}{1 + 869e^{-1.33t}} \right\rfloor \quad (23)$$

for $0 \leq t \leq 11$ where t is months. The floor function $\lfloor \cdot \rfloor$ is in the math module named `math.floor()`. The e symbol means exponentiation operation, this is available from `math.exp()` function in math module.

For example,

$$hh(0) = 0 \text{ ducks} \quad (24)$$

$$hh(5) = 250 \text{ ducks} \quad (25)$$

$$hh(10) = 531 \text{ ducks} \quad (26)$$

Deliverables Problem 8

- The math module is already imported in a1.py.
- Complete the *hh* function in the file a1.py.
- Round your answer to the nearest integer using `math.floor()`

Problem 9: Throwing a Stone

A stone is thrown straight up from the roof of an 80 ft building with an initial velocity of 64 ft/sec. The height (in feet) of the stone at any time t seconds is given by:

$$\text{height}(t) = -16t^2 + 64t + 80 \quad (27)$$

The rock will hit the ground after 5 seconds:

$$\text{height}(5) = 0 \text{ feet} \quad (28)$$

Deliverables Problem 9

- Complete the *height* function in the file a1.py
- Round the answer to 2 decimal places.

Problem 10: Treating Heart Attacks

According to the American Heart Association, the treatment benefit for heart attacks depends on the time (hours) until treatment and is described by:

$$B(t) = \frac{0.44t^4 + 700}{0.1t^4 + 7} \quad (29)$$

for $(0 \leq t \leq 24)$

Deliverables Problem 10

- Complete the *B* function in the file a1.py.
- For this problem, you'll need to make your own input/outputs.
- Round the answer to 2 decimal places.

Problem 11: Roots to the Quadratic

Recall that a quadratic is a function:

$$q(x) = ax^2 + bx + c \quad (30)$$

A root is a number that makes the function zero. For example, if

$$q(x) = 2x^2 + 5x - 12 \quad (31)$$

then the two roots are $x = -4$ and $x = 3/2$:

$$q(-4) = 2(-4)^2 + 5(-4) - 12 = 32 - 20 - 12 = 0 \quad (32)$$

$$q(3/2) = 2(3/2)^2 + 5(3/2) - 12 = 2(9/4) + 15/2 - 12 = (24/2) - 12 = 0 \quad (33)$$

Implement a function `quad(a,b,c,x)` that returns `True` if x is a root for $ax^2 + bx + c$ and `False` otherwise. Running the function:

```
1 print(quad(2,5,-12,-4))
2 print(quad(2,5,-12,3/2))
3 print(quad(2,5,-12,1))
```

gives output:

```
1 True
2 True
3 False
```

Deliverables Problem 11

- Complete the `quad` function in the file `a1.py`

Problem 12: Sinking Fund

This model describes a “sinking fund.” Money is periodically set aside until a date is reached. One important kind is retirement—money that will be available when you stop working. You must know the payment amount P , the number of times a year you make the payment n , the number of years you make the payment t , and the interest rate r . For example,

- If you pay \$22,000 once a year for seven years that has 6% compounded annually, you’ll have, at the end \$184,664.43.
- If you make \$500 monthly that has 4% (about the current best rate) compounded monthly for 20 years, you’ll have \$183,387.31
- If \$1,200 is deposited quarterly with 8% compounded quarterly for 10 years, you’ll have \$72,482.38

The sinking fund is:

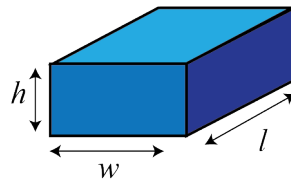
$$R(P, r, n, t) = P \left[\frac{\left(1 + \frac{r}{n}\right)^{nt} - 1}{\frac{r}{n}} \right] \quad (34)$$

Deliverables Problem 12

- Complete the R function in the file a1.py
- Round to two decimal places.

Problem 13: Rectangular Solid

Complete the Python function that gives the total surface area S of a rectangular solid with dimensions w, l, h shown here:



The surface area is:

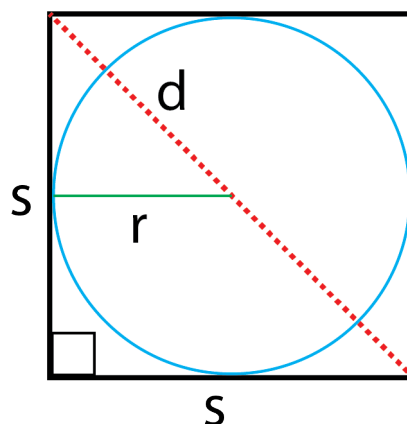
$$S(w, l, h) = (?) \quad (35)$$

Deliverables Problem 13

- Complete the S function in the file a1.py
- You must create your own solutions for this problem.

Problem 14: Geometry

You are given two functions to implement. The first takes the length of a side of a square and returns the diagonal (no units). The second takes a diagonal of a square and returns the area of the largest circle that can be inscribed in the square. A figure is shown below.



The side is s , the diagonal (red dotted line) is d , the radius (green) is r , and the circle blue. The easiest solution involves using the Pythagorean Theorem. Here is a run:

```
1 s = 10
2 a = circle_area(square_diagonal(s))
3 print(a)
```

has output:

```
1 78.54
```

where we have rounded to two decimal places. As you can see, there is function composition: the output of the first function is the input to the second.

Deliverables Problem 14

- Complete the two functions for this problem in the file `a1.py`
- You can create some of your own tests to verify your implementation is correct.
- Round to two decimal places.

Problem 15: Toward Sports Analytics

A pitcher's earned-run average written ERA is determined by:

$$\text{ERA}(e, i, t) = \left(\frac{e}{i}\right)t \quad (36)$$

Here, e is earned runs (not the exponentiation operation), i is innings pitched, and t is total innings. For example, if a pitcher had 4 earned runs, pitched 6 innings, and the total number of innings was 9, then the ERA is:

$$\text{ERA}(e, i, t) = \left(\frac{4}{6}\right)9 = 6 \quad (37)$$

Deliverables Problem 15

- Complete the ERA function in file `a1.py`
- You can create some more test yourself to verify your implementation is correct.
- Round to two decimal places.

Problem 16: Climate

Wind chill is a somewhat vague phenomenon associated with the perception of temperature drop that a person feels. There does not exist any standard for wind chill, but in the U.S., the wind chill index T_{wc} is:

$$T_{wc} = 35.74 + 0.6215T_a - 35.75v^{0.16} + 0.4275T_av^{0.16} \quad (38)$$

For example, if the temperature is 2°C and wind velocity is 5 mph, then the wind chill is -9°C .

Deliverables Problem 16

- Complete the `T_wc` function in file `a1.py`
- Create some of your own tests to verify your implementation is correct.
- Use the `math` floor function on the value.

Problem 17: Guessing (Approximation)

Most of the time we cannot easily calculate a value, *e.g.*, too big, too small, too complicated. In computing we have learned to guess (or what we call *approximate*). You're no doubt familiar with factorial $n!$, but did you know it was known in the 18th century? Without computers, calculating $n!$ for even what we would consider tiny values was quite difficult. An eponymous approximation, called the Stirling approximation, is (as a lower bound):

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \quad (39)$$

While this might actually look more difficult it isn't. The following code:

```
1 n0 = 10
2 print(math.factorial(n0),fact_est(n0))
3 n0 = n0 * 10
4 print(math.factorial(n0),fact_est(n0))
```

produces:

```
1 3628800 3598695
2
3 93326215443944152681699238856266700490715968264381621
4 46859296389521759999322991560894146397615651828625369
5 7920827223758251185210916864000000000000000000000000000
6
```

```
7 93248476252694195831713952173205863434091892055096263
8 58057629519885644096343253465550010174734371150486577
9 9454898420101234457113230028869790337578478524694528
```

When you run this, it'll look different—because the numbers have so many digits, the values don't easily lend themselves to print.

Deliverables Problem 17

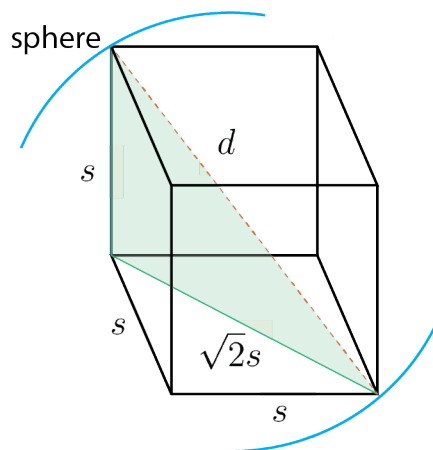
- Complete the `fact_est()` function in file `a1.py`
- Please create some additional test to verify your implementation is correct.
- Return the approximation as an integer (cast it as an `int`).

Problem 18: Constraints

Given a sphere's volume, v , determine the length of the side of the largest cube inscribed. To help us, we recall the formula for the volume of a sphere:

$$v(r) = \left(\frac{4}{3}\right)\pi r^3 \quad (40)$$

We sketch out the cube that includes the sphere's diameter d and sides s of the cube:



Implement two functions `volume_to_radius(v)` that takes the volume of a sphere and gives the radius and `side_max_square(v)` that takes the volume of a sphere and returns the side of the largest cube inscribed. This second function uses the first function. The following code:

```
1 print(volume_to_radius(v))
2 print(side_max_square(v))
```

produces:

1 8.0
2 9.24

Deliverables Problem 18

- Complete the functions `volume_to_radius(v)` and `side_max_square(v)` in file `a1.py`
- Round the return value of `volume_to_radius(v)` and `side_max_square(v)` to 2 decimal places.

Problem 19: Investment Strategy

A simple investment strategy is to fix a dollar amount to purchase a stock; this would be called a *time diversification* strategy. The idea is that, over time, the price should rise following the smaller purchase price. Here's some data:

\$ Investement	Market \$ Price/Share	Shares
1000	40	250
1000	35	286
1000	34	294
1000	38	263
1000	50	200

that reflects five time periods. To find the average price per share of a sequences of market price per shares $m_1, m_2, \dots, m_n$, we use the formula:

$$app(m_1, m_2, \dots, m_n) = \frac{1}{n} \sum_{i=1}^n m_i = \frac{1}{n} (m_1 + m_2 + \dots + m_n) \quad (41)$$

Python has a function `sum()` that takes either a list or tuple and returns the sum:

```
1 >>> x = [1, 2, 3, 4]
2 >>> y = (1, 2, 3, 4)
3 >>> sum(x), sum(y)
4 (10, 10)
5 >>> len(x), len(y)
6 (4, 4)
7 >>>
```

Recall that `len()` gives the number of members in the list or tuple. Implement a function `app(market)` that takes a list of the market price per shares and returns the tuple `(x, y)`

where x is the average price per share (rounded to two decimal places) and y is the last market price so that we can compare the two. If $x < y$ then your strategy has worked. The following code:

```
1 market = [40,35,34,38,50]
2 print(app(market))
```

produces:

```
1 (39.4, 50)
```

Problem 20: Patterns and Occam's Razor

One of the problems with using a few examples to describe a function is that there are a virtually infinite number of satisfying expressions. A pattern helps us understand and make decisions on data, but if there are competing explanations, which one do we choose? Here's a simple example. We stated in lecture that the simplest description of a function is the first several values:

$$(x_0, y_0), (x_1, y_1), (x_2, y_2), \dots \quad (42)$$

Suppose you're faced with this pattern:

$$(1, 1), (2, 16), (3, 81), (4, 256), \dots \quad (43)$$

One of your employees has this function:

$$\text{model1}(n) = 10n^3 - 35n^2 + 50n - 24 \quad (44)$$

You believe there's a more intuitive model. Complete the functions:

- `model1(n)` that uses the employee's function
- `model2(n)` that uses your function
- `model_compare(inputs)` where `inputs = (1,2,3,4)` and returns a tuple of five tuples: `((x0,model1(x0),model2(x0)), (x1,model1(x1),model2(x1)), (x2,model1(x2),model2(x2)), (x3,model1(x3),model2(x3)), ((5,model1(5), model2(5)))` where we've split the tuple initially:

```
1 x0,x1,x2,x3 = inputs
```

Remember that Python's default structure is a tuple. The following code:

```
1 inputs = (1,2,3,4)
2 print(model_compare(inputs))
```

produces:

```
1 ((1, 1, 1), (2, 16, 16), (3, 81, 81), (4, 256, 256), (4, 601, 625))
```

Given explanations, we often seek to use the simplest. This strategy is called “Occam’s razor” (please visit https://en.wikipedia.org/wiki/Occam%27s_razor).

Communication

When connecting with a new person, you should introduce yourself, give a short message describing the purpose, then salutation. Please use and adapt this format when connecting to a new partner. The student pairs are given on the next page.

Dear Student,

My name is X Y and I'm a student in your C200, but a different section. It looks like we're partners this week. I'm generally free on Thursday, Saturday, Sunday. I'm hoping we can knock most of this out on our first meeting. Luddy is a great place if we can snag a conference room.

Take care,

Another Student

Format for InScribe post

Hi Instructor/ TA <Whom ever they want to ask question/ Share something>,

If you are posting about lecture/Notes

My name is Student-A. I am in Dr. (mention faculty instructor name) lecture section. This is regarding

If you are posting about labs

My name is Student-A. I am in the lab section that meets on DAY, TIME of your lab and NAME OF THE TA.
This is regarding

Thanks and regards,

<Name>

<User name>

Programming partners

coubenne@iu.edu, marcmarq@iu.edu
talfares@iu.edu, timolaws@iu.edu
ejaini@iu.edu, liaobrie@iu.edu
efoltyn@iu.edu, anryoun@iu.edu
egilmour@iu.edu, adivaidy@iu.edu
dkembert@iu.edu, hailreid@iu.edu
btcunnin@iu.edu, taylbryo@iu.edu
blbinder@iu.edu, bobbyun@iu.edu
ethdeatr@iu.edu, cpmathew@iu.edu
chfurlow@iu.edu, sholove@iu.edu
coobratt@iu.edu, wschrama@iu.edu
goldfusl@iu.edu, westje@iu.edu, mzuhair@iu.edu
ckbarts@iu.edu, klohiya@iu.edu
grhamlin@iu.edu, kroussel@iu.edu
skalluru@iu.edu, rasahu@iu.edu
nimjain@iu.edu, whitedyl@iu.edu
aabdulw@iu.edu, negia@iu.edu
addfishe@iu.edu, jwubelho@iu.edu
rdotsu@iu.edu, hsh1@iu.edu
agahlau@iu.edu, froy@iu.edu
mahchaud@iu.edu, smungee@iu.edu
byersjac@iu.edu, kyalane@iu.edu
nicdelg@iu.edu, mmaplet@iu.edu
garmenda@iu.edu, matyun@iu.edu
colguth@iu.edu, jjrinald@iu.edu
kevgross@iu.edu, evanmayo@iu.edu
banksda@iu.edu, mszach@iu.edu
agotanco@iu.edu, lvumpa@iu.edu
ttfender@iu.edu, jewach@iu.edu
aare@iu.edu, thebmill@iu.edu
keswa@iu.edu, cardnorr@iu.edu
dayinla@iu.edu, elwesley@iu.edu
bdgonzal@iu.edu, kiloscot@iu.edu
bbeer@iu.edu, aidsteph@iu.edu
abondada@iu.edu, zs15@iu.edu
twchase@iu.edu, mtay@iu.edu
jkuk@iu.edu, lukthiel@iu.edu
jahjacks@iu.edu, pmiesch@iu.edu

xbchrist@iu.edu, alejmora@iu.edu
bthenson@iu.edu, roberjuv@iu.edu
jaeblank@iu.edu, kr15@iu.edu
okirchen@iu.edu, vpottete@iu.edu
ojallow@iu.edu, antzamor@iu.edu
cgcheane@iu.edu, rosanghv@iu.edu
jk314@iu.edu, cmolaski@iu.edu
sabyrap@iu.edu, mt87@iu.edu
kadlakha@iu.edu, bogawa@iu.edu
mjfarrin@iu.edu, bawsung@iu.edu
ldeel@iu.edu, ansjmeht@iu.edu
crigonza@iu.edu, ksabloak@iu.edu
dajuhnke@iu.edu, omwells@iu.edu
nitiarun@iu.edu, atsarver@iu.edu
evarcarm@iu.edu, tersacks@iu.edu
jejea@iu.edu, kokulski@iu.edu
benchart@iu.edu, cjrodenb@iu.edu
bahls@iu.edu, risvphil@iu.edu
nganesh@iu.edu, gsskippe@iu.edu
gkreutes@iu.edu, ydpatil@iu.edu
tgebrewo@iu.edu, dmw6@iu.edu
alehe@iu.edu, prnemani@iu.edu
nafike@iu.edu, esspatel@iu.edu
langarbe@iu.edu, dmonroyh@iu.edu
cmfalk@iu.edu, robreese@iu.edu
jagoodal@iu.edu, isapittm@iu.edu
enkarl@iu.edu, sumuth@iu.edu
carthugh@iu.edu, emasseng@iu.edu
rbernfel@iu.edu, sxanders@iu.edu
elscheng@iu.edu, tanilitt@iu.edu
bgelard@iu.edu, ozsachs@iu.edu
cjo3@iu.edu, tkthang@iu.edu
jamcham@iu.edu, sandro@iu.edu
lddial@iu.edu, prschulz@iu.edu
joglickm@iu.edu, sstickl@iu.edu
boydmad@iu.edu, zz67@iu.edu
ebederma@iu.edu, msharsh@iu.edu
evbutts@iu.edu, cnshimi@iu.edu
zahaider@iu.edu, lisalaza@iu.edu
mrb9@iu.edu, jrwaren@iu.edu

mbenites@iu.edu, jquansah@iu.edu
smhamrah@iu.edu, martibr@iu.edu
cdasari@iu.edu, lemarc@iu.edu
hdjour@iu.edu, aramanu@iu.edu
skapure@iu.edu, mclindbe@iu.edu
gibsonn@iu.edu, cammlanc@iu.edu
yc127@iu.edu, gsackie@iu.edu
eberens@iu.edu, alzemlya@iu.edu
dbarbari@iu.edu, nmeely@iu.edu
mthinman@iu.edu, kalovert@iu.edu
clarkina@iu.edu, conaitis@iu.edu
adodaro@iu.edu, youngct@iu.edu
kbussel@iu.edu, zhangsuo@iu.edu
lafly@iu.edu, dematt@iu.edu
jkittur@iu.edu, cjlomax@iu.edu
rkoston@iu.edu, rvarrier@iu.edu
drmiguth@iu.edu, jlaytin@iu.edu
ajafowl@iu.edu, mmalviya@iu.edu
elehaged@iu.edu, zanmille@iu.edu
adekolaw@iu.edu, hshyama@iu.edu
cedrchen@iu.edu, mitcal@iu.edu
ahmealsh@iu.edu, bsati@iu.edu
armstan@iu.edu, wl52@iu.edu
rugaddam@iu.edu, ssp4@iu.edu
jaccryst@iu.edu, alniehau@iu.edu
brjduck@iu.edu, kzerbino@iu.edu
drussow@iu.edu, manpate@iu.edu
mugabr@iu.edu, prmundra@iu.edu
pranchan@iu.edu, dommonte@iu.edu
radutta@iu.edu, ozopich@iu.edu
sdondeti@iu.edu, fsafianu@iu.edu
shkani@iu.edu, angzamora@iu.edu
concraft@iu.edu, jasreese@iu.edu
mkulekci@iu.edu, natkwon@iu.edu
ahsans@iu.edu, ishyadav@iu.edu
rogoenka@iu.edu, missavin@iu.edu
saskenaz@iu.edu, dyllopez@iu.edu
xmdrew@iu.edu, rysalina@iu.edu
yaljabri@iu.edu, nmenne@iu.edu
damaatki@iu.edu, isundara@iu.edu

vbhattad@iu.edu, aralover@iu.edu
karbern@iu.edu, adenyu@iu.edu
jb108@iu.edu, aluckhau@iu.edu
nantinao@iu.edu, sasubb@iu.edu
moazimi@iu.edu, isvoor@iu.edu
ag38@iu.edu, jmsequei@iu.edu
chjflor@iu.edu, bgtruong@iu.edu
emhjames@iu.edu, fasrasoo@iu.edu
ljhorman@iu.edu, mloko@iu.edu
jdkakare@iu.edu, isshetty@iu.edu
laigrant@iu.edu, marsh1@iu.edu
ag46@iu.edu, aamuse@iu.edu
michdool@iu.edu, shamonei@iu.edu
bowenbm@iu.edu, irodeman@iu.edu
jergalan@iu.edu, mawlorch@iu.edu
tycarmo@iu.edu, rvenigal@iu.edu
ghargro@iu.edu, maymat@iu.edu
nfarzane@iu.edu, ijrussel@iu.edu
srkagama@iu.edu, adimaran@iu.edu
bboyapat@iu.edu, ainlukas@iu.edu
ssdatla@iu.edu, jakpatel@iu.edu
salvchri@iu.edu, aposton@iu.edu
mgbekele@iu.edu, sunathan@iu.edu
tbashee@iu.edu, kalerobi@iu.edu
zalsamri@iu.edu, jakepars@iu.edu
adguhan@iu.edu, nieywill@iu.edu
pikhatri@iu.edu, pp22@iu.edu
owflynn@iu.edu, pjminne@iu.edu
owencape@iu.edu, cnorcutt@iu.edu
nifredri@iu.edu, mjo4@iu.edu
ajkoutsu@iu.edu, joevu@iu.edu
ehchoo@iu.edu, ctsvetan@iu.edu
hayburge@iu.edu, wkpierce@iu.edu
agrief@iu.edu, amarynow@iu.edu
fcohan@iu.edu, ayorgen@iu.edu
bedani@iu.edu, olesmith@iu.edu
bskendal@iu.edu, jo16@iu.edu
rkoranne@iu.edu, audscha@iu.edu
asgurram@iu.edu, clushell@iu.edu
btbasing@iu.edu, parishar@iu.edu

kkunshet@iu.edu, anangla@iu.edu
tbrose@iu.edu, nguyen17@iu.edu
zkabacin@iu.edu, vnathany@iu.edu
dh43@iu.edu, jopantoj@iu.edu
reclay@iu.edu, jawegman@iu.edu
keswar@iu.edu, smitfi@iu.edu
mdevara@iu.edu, rpolu@iu.edu
florech@iu.edu, ibshaw@iu.edu
leidaugh@iu.edu, kw125@iu.edu
mitja@iu.edu, gl25@iu.edu
clickc@iu.edu, jskains@iu.edu
abeathea@iu.edu, camoverm@iu.edu
sergonza@iu.edu, cmekat@iu.edu
natkling@iu.edu, cmellgre@iu.edu
sydforem@iu.edu, shwanara@iu.edu
heissler@iu.edu, hmahapat@iu.edu
idasilva@iu.edu, rnalluri@iu.edu
akeshav@iu.edu, jl367@iu.edu
aarepal@iu.edu, vpotluri@iu.edu
dheer@iu.edu, kenmoses@iu.edu
pchitwoo@iu.edu, braxluns@iu.edu
edeitchl@iu.edu, davepais@iu.edu
jigarnes@iu.edu, anrajput@iu.edu
bllfost@iu.edu, jymenach@iu.edu
doandre@iu.edu, drimawi@iu.edu
jbriar@iu.edu, reshetty@iu.edu
andedaws@iu.edu, cpmann@iu.edu
maboncos@iu.edu, everma@iu.edu
adhimat@iu.edu, lydiwang@iu.edu
nwconn@iu.edu, ew38@iu.edu
eabilius@iu.edu, vivitran@iu.edu
jackchap@iu.edu, rvasude@iu.edu
akoduru@iu.edu, cs155@iu.edu
iblay@iu.edu, nealsing@iu.edu
rpkeatin@iu.edu, stewrach@iu.edu
vkabra@iu.edu, roldenbu@iu.edu
iyersan@iu.edu, aml4@iu.edu
blstcoop@iu.edu, athason@iu.edu
tborder@iu.edu, anupand@iu.edu
ethhanna@iu.edu, samyadav@iu.edu

heoji@iu.edu, mjseymou@iu.edu
bcarpine@iu.edu, djtheodo@iu.edu
nikdevra@iu.edu, muksingh@iu.edu
lajith@iu.edu, aidpride@iu.edu
abrking@iu.edu, smitaa@iu.edu
mailic@iu.edu, neenpate@iu.edu
augguy@iu.edu, tnerkar@iu.edu
felkhatt@iu.edu, sahasrir@iu.edu
herncr@iu.edu, jw363@iu.edu
dgiffor@iu.edu, navann@iu.edu
shkakk@iu.edu, jnimmala@iu.edu
rgyasini@iu.edu, slopezve@iu.edu
jbosio@iu.edu, tlichten@iu.edu
rycostin@iu.edu, ndmyer@iu.edu
vicbarre@iu.edu, mkreddy@iu.edu
adj18@iu.edu, seuryu@iu.edu
ramgowda@iu.edu, mmmatias@iu.edu
mahichan@iu.edu, rheashah@iu.edu
ealfalah@iu.edu, antonegr@iu.edu
sanjais@iu.edu, jhl14@iu.edu
alneyadk@iu.edu, soyedele@iu.edu
adatwani@iu.edu, andyni@iu.edu
dqchoe@iu.edu, jacdstev@iu.edu
nbriere@iu.edu, dajshaw@iu.edu
mabana@iu.edu, ncw1@iu.edu
ndodoh@iu.edu, auskirvi@iu.edu
alurs@iu.edu, mahonm@iu.edu
am280@iu.edu, aishravi@iu.edu
skyangel@iu.edu, kevwong@iu.edu
shkami@iu.edu, tunnitha@iu.edu
mgoenka@iu.edu, swaschow@iu.edu
etferba@iu.edu, abbywebe@iu.edu
meiyhe@iu.edu, nshnoble@iu.edu
svdatla@iu.edu, olsoncs@iu.edu
ldeveau@iu.edu, dslipch@iu.edu
bronburk@iu.edu, corawang@iu.edu
mfbah@iu.edu, ssanjee@iu.edu
dakomi@iu.edu, jacloise@iu.edu
ygajjar@iu.edu, kpazdur@iu.edu
adagar@iu.edu, vpremku@iu.edu

eynkansa@iu.edu, jiajtang@iu.edu
jjithesh@iu.edu, nikhmorg@iu.edu